

Project Report: Python Banking
Simulation System



VIT[®]

BHOPAL

Kinjal Agrawal

26BCE10768

Introduction to problem solving and
programming CSE 1021

Priyadharshini B. maam

Introduction

Banking systems are among the most familiar real-world software applications, making them an ideal subject for demonstrating fundamental programming concepts. This project implements a console-based Banking Simulation System in Python, allowing a user to create an account, authenticate with a PIN, and perform core operations – balance inquiry, deposit, withdrawal, and statement generation – entirely through core language constructs taught in this course: variables, conditionals, loops, functions, dictionaries, and lists.

Problem Statement

Manually tracking bank account transactions is

error-prone and offers no structured way to validate operations such as withdrawals against an available balance. This project addresses that gap by simulating a simplified banking system in software, where every operation is validated and logged programmatically, ensuring accurate and reliable transaction processing.

Functional Requirements

The system is built around three major functional modules:

1. Account Management — create an account (name, initial balance, 4-digit PIN); authenticate the user via PIN before granting access to any operation, check current balance.

2. Transaction Processing — deposit funds and withdraw funds, both with input validation (rejecting negative,

zero, or non-numeric amounts) and an overdraft check on withdrawal.

3. Reporting – mini statement (last 5 transactions), full transaction history, and a session summary (total deposited, total withdrawn, net change, closing balance).

Each module has a clear input/output structure. The user is prompted for input via the console, the relevant module validates and processes it, and a confirmation or error message is printed back. The overall workflow is menu-driven – after authentication, the user repeatedly selects an option from a numbered menu until choosing to exit.

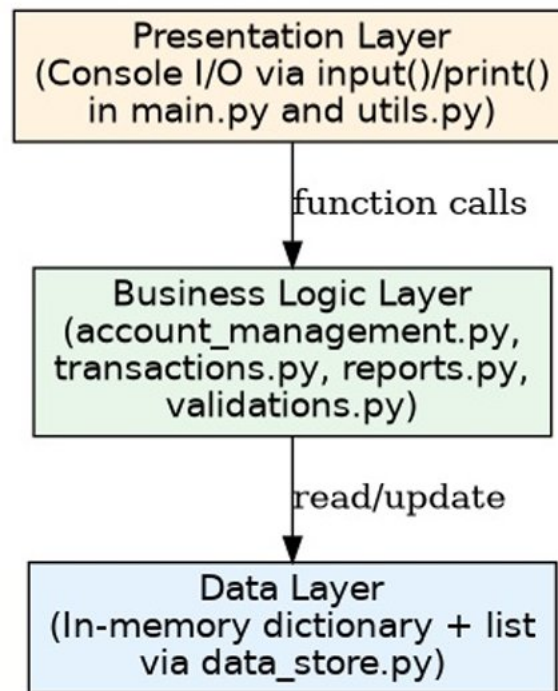
Non-Functional Requirements

<u>Requirement</u>	<u>How It Is Addressed</u>
Performance	All account operations (balance check, deposit, withdrawal) run in constant

	<i>time ($O(1)$) since account data is stored in a dictionary; reporting operations run in $O(n)$ over the transaction list, which is efficient for a single-session application.</i>
<i>Security</i>	<i>Access to account operations requires PIN authentication, with a maximum of 3 attempts before the session locks.</i>
<i>Usability</i>	<i>A clear, numbered menu is displayed after every action; all prompts explicitly state what input is expected, and error messages explain exactly what went wrong.</i>
<i>Reliability</i>	<i>Every transaction updates the balance and the transaction log together, so the two never fall out of sync within a session.</i>
<i>Error Handling Strategy</i>	<i>Dedicated validation functions (validations.py) reject invalid input before it reaches business logic; a top-level try/except in main.py catches any unexpected error without crashing the session.</i>
<i>Maintainability</i>	<i>The codebase is split into 7 single-purpose modules (data, validation, 3 functional modules, UI, entry point), making each piece easy to test, debug, and extend independently.</i>

System Architecture

The system follows a simple 3-layer architecture:

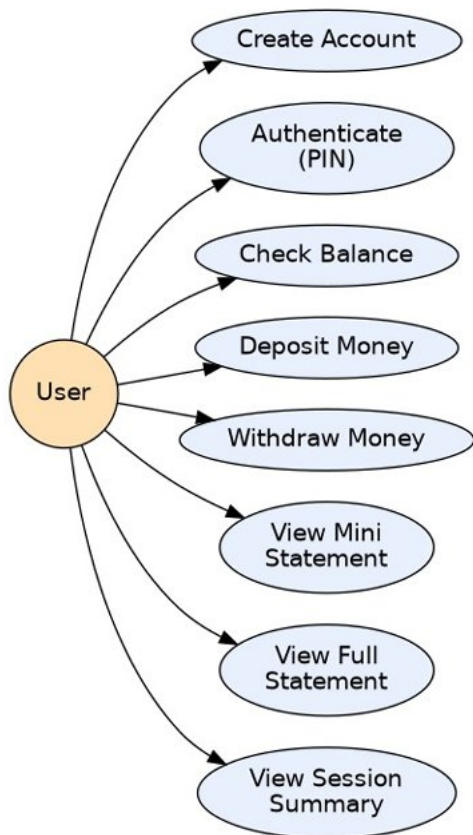


System Architecture

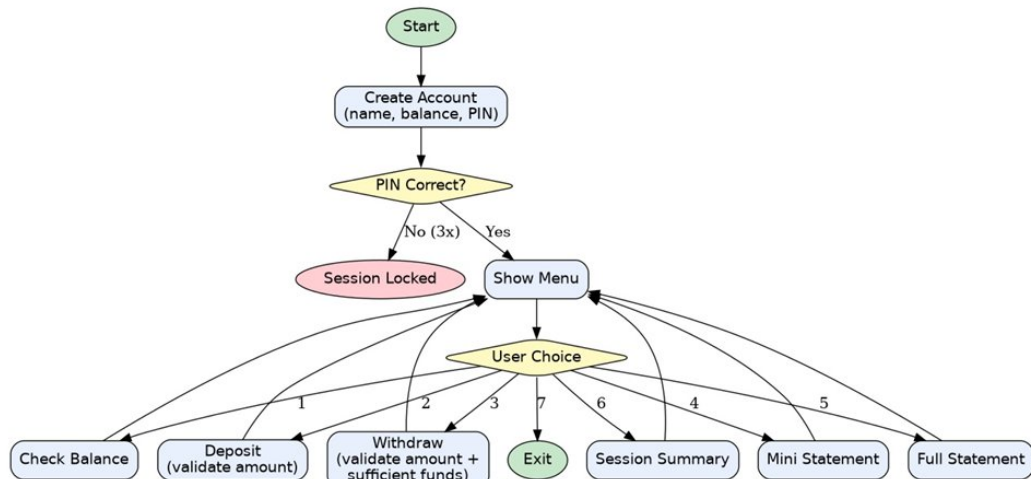
- *Presentation Layer – handles all console input/output (main.py, utils.py)*
- *Business Logic Layer – implements the three functional modules and validation (account_management.py, transactions.py, reports.py, validations.py)*
- *Data Layer – the in-memory account dictionary and transaction list (data_store.py)*

Design Diagrams

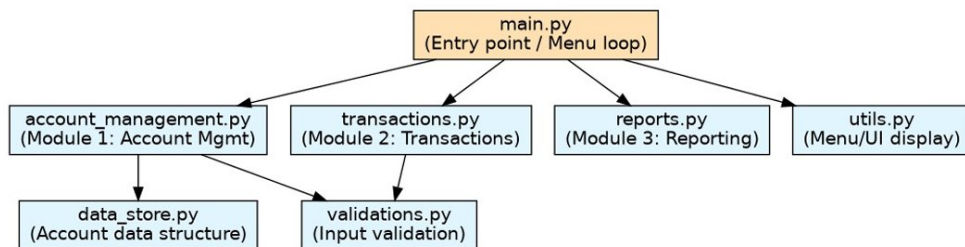
Use Case Diagram:



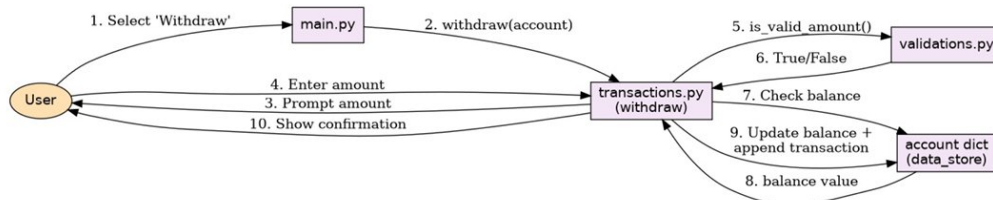
Workflow / Process Flow Diagram



Component Diagram



Sequence Diagram



Design Decisions & Rationale

- **Dictionary for account data:** chosen over separate variables because it groups all related fields (name, balance, PIN, transactions) into a single structure that can be passed between functions cleanly.
- **List for transaction history:** chosen because transactions naturally form an ordered sequence, and Python lists

support efficient appending and slicing (e.g., retrieving the last 5 entries for the mini statement).

- **No classes/OOP:** *the project deliberately uses only functions and basic data structures, matching the “basic Python” scope of the course at this stage, rather than introducing object-oriented concepts not yet covered.*
- **Modular file split:** *rather than one large script, logic is divided into 7 files by responsibility, directly satisfying the project’s maintainability and clean-implementation requirements.*
- **PIN authentication:** *added beyond the original scope to meaningfully satisfy the “Security” non-functional requirement, since a banking system without any access control would be unrealistic.*

Implementation Details

Each functional module operates on this dictionary, which is passed as a parameter between functions rather than relying on global state, keeping functions independently testable. Input validation is applied before any value is written to the account, and every successful deposit or withdrawal appends a human readable string to the transactions list, which the reporting module later reads to generate statements and the session summary.

RESULTS:

```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
= RESTART: C:\Users\Kinjal Agrawal\OneDrive\Scans\Pictures\Documents\Desktop\CSE1021\src\FINAL PROJECT.py
=====
Welcome to the Python Banking Simulation System
=====

----- Create New Account -----
Enter account holder name: Kinjal
Enter initial deposit amount: 500
Set a 4-digit PIN for your account: 1234

Account created successfully for Kinjal!
Initial Balance: 500.00

Enter your 4-digit PIN to continue: 1234
Authentication successful.

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 2
Enter amount to deposit: -50

Invalid input. Please enter a valid positive number.

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 2
Enter amount to deposit: abc

Invalid input. Please enter a valid positive number.

===== BANKING SIMULATION MENU =====
1. Check Balance
Ln: 124 Col: 0

25°
29-09-2026 22:38
ENG IN

IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
7. Exit
=====
Enter your choice (1-7): 2
Enter amount to deposit: abc

Invalid input. Please enter a valid positive number.

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 3
Enter amount to withdraw: 200

Withdrawal successful! New Balance: 300.00

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 3
Enter amount to withdraw: 900

Insufficient funds! Withdrawal denied.

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 6

----- Session Summary -----
Ln: 124 Col: 0

25°
29-09-2026 22:38
ENG IN
```

```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help

1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 6

----- Session Summary -----
Total Deposited : 0.00
Total Withdrawn : 200.00
Net Change      : -200.00
Closing Balance : 300.00
-----

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 4

----- Mini Statement -----
1. Withdraw: -200.00
-----

===== BANKING SIMULATION MENU =====
1. Check Balance
2. Deposit
3. Withdraw
4. Mini Statement
5. Full Transaction History
6. Session Summary
7. Exit
=====
Enter your choice (1-7): 7

Thank you for banking with us, Kinjal!
Session ended.

>>> |
```

Ln: 124 Col: 0

25° ENG IN 22:38 29-09-2026

```
IDLE Shell 3.14.7
File Edit Shell Debug Options Window Help
Python 3.14.7 (tags/v3.14.7:823f032, Aug 5 2026, 10:51:32) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:\Users\Kinjal Agrawal\OneDrive\Scans\Pictures\Documents\Desktop\CSE 1021\src\FINAL PROJECT.py
=====
Welcome to the Python Banking Simulation System
=====

----- Create New Account -----
Enter account holder name: kinjal
Enter initial deposit amount: 1345
Set a 4-digit PIN for your account: 1234

Account created successfully for kinjal!
Initial Balance: 1345.00

Enter your 4-digit PIN to continue: 2222
Incorrect PIN. Attempts remaining: 2
Enter your 4-digit PIN to continue: 4444
Incorrect PIN. Attempts remaining: 1
Enter your 4-digit PIN to continue: 3333
Incorrect PIN. Attempts remaining: 0
Too many incorrect attempts. Session locked.
>>> |
```

Testing Approach

<u>Test Case</u>	<u>Input</u>	<u>Expected Result</u>	<u>Result</u>
<u>Valid deposit</u>	500	Balance increases by 500	Pass
<u>Invalid deposit(negative)</u>	-50	error	Pass
<u>Invalid deposit(text)</u>	abc	Error	Pass
<u>Valid withdrawal</u>	200	Balance decreases by 200	Pass
<u>Overdraft withdrawal</u>	999	error	Pass
<u>Correct PIN</u>	4 digit pin	Authentication successful	Pass
<u>Incorrect PIN (3x)</u>	Non matching pin	Session locked	Pass

Challenges Faced

- Deciding how to validate numeric input without using external libraries ,resolved using basic string methods
(.isdigit(), .replace()) instead of try/except float() conversions, to keep validation logic simple and explicit.
- Keeping the design meaningfully modular (5-10 files) without artificially splitting logic that belongs together ,resolved by aligning each file

with exactly one functional module or responsibility.

- *Representing “Security” and “Logging” in a project without a real authentication backend , resolved with a simple but genuine PIN check mechanism and an in-memory transaction log.*

Learnings & Key Takeaways

This project reinforced how foundational programming concepts ,conditionals, loops, functions, and basic data structures are sufficient to build a functionally complete, realistic application when combined with careful validation and modular design. It also demonstrated how non functional requirements (security, reliability, maintainability) can be addressed meaningfully even in a small-scale project, not just in large systems.

Future Enhancements

- *Persistent storage (e.g., saving account data to a file or database) so data survives between sessions*

- *Support for multiple accounts with individual login credentials Fund transfers between accounts .*
- *Interest calculation for savings accounts A graphical user interface instead of a console interface.*

END